Escola Tècnica Superior d'Enginyeria de Telecomunicació de Barcelona
UNIVERSITAT POLITÈCNICA DE CATALUNYA

Estructures de dades dinàmiques

Programació i Estructures de Dades

Grau en Enginyeria Electrònica de Telecomunicació

ETSETB Telecom BCN

Introducció

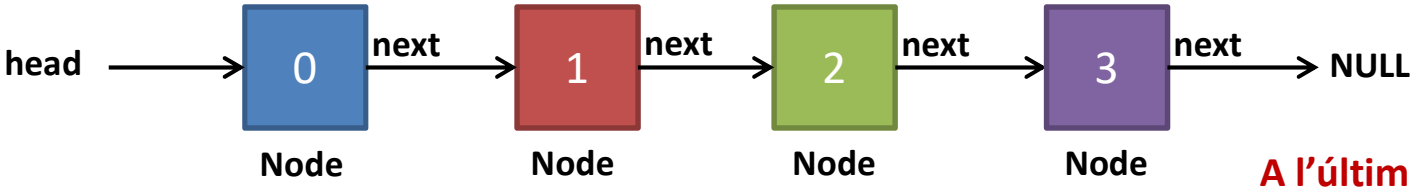
- Una **estructura de dades** és una **forma d'organitzar dades a la memòria de l'ordinador** perquè puguin ser manipulades eficientment
  - Existeixen **molts tipus d'estructures de dades**, cobrint **diferents necessitats específiques**
  - Tan **important** és l'**elecció de les sentències que defineixen un bon algorisme** com l'**organització en memòria de les dades**!
- Podem diferenciar entre **estructures de dades estàtiques i dinàmiques**:
  - **Estàtiques**: la seva mida es defineix abans que el programa s'executi i no varia durant la seva execució (p.e., vectors, structs)
  - **Dinàmiques**: la seva mida pot variar durant l'execució del programa, segons les necessitats del moment (p.e., llistes)

Llistes basades en array

- **Les llistes són estructures de dades dinàmiques** que **emmagatzemen una seqüència ordenada d'elements**, **adaptant la seva capacitat** en funció del nombre d'elements que guarden
- Les llistes **basada en array** (***array list***) són capaces d'adaptar la seva capacitat en les operacions d'afegir/inserir/eliminar elements:
  - **Molt eficients en obtenir un element donada la seva posició (sempre en temps constant)**, doncs els elements dins l'array intern es guarden en posicions contigües de memòria ☺
  - **Poc eficients en les operacions d'afegir/inserir/eliminar elements**, requerint redimensionats del seu array intern i desplaçaments d'elements cap a la dreta i esquerra ☹

Llista enllaçada (1/4)

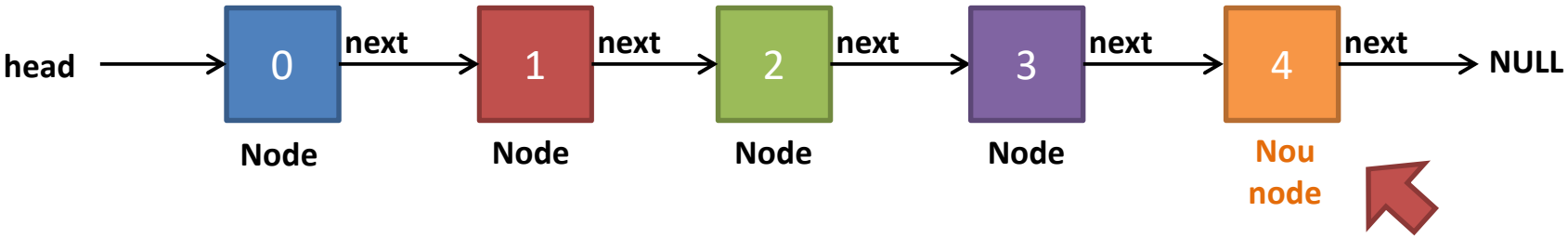
- Una **forma alternativa d'implementar una estructura de dades de tipus llista** és la llista enllaçada (*linked list*, en anglès)
- L'**objectiu principal** de les llistes enllaçades és **millorar l'eficiència a l'hora d'afegir/inserir/eliminar elements**, l'aspecte millorable de les *array list*
- Una llista enllaçada està **formada per nodes**, encarregats de desar **les dades a la llista** (cada node representa una posició de la llista). A més, **cada node guarda un punter al següent node de la llista**:



A l'últim node de la llista, next = NULL, doncs no hi ha pas cap node següent

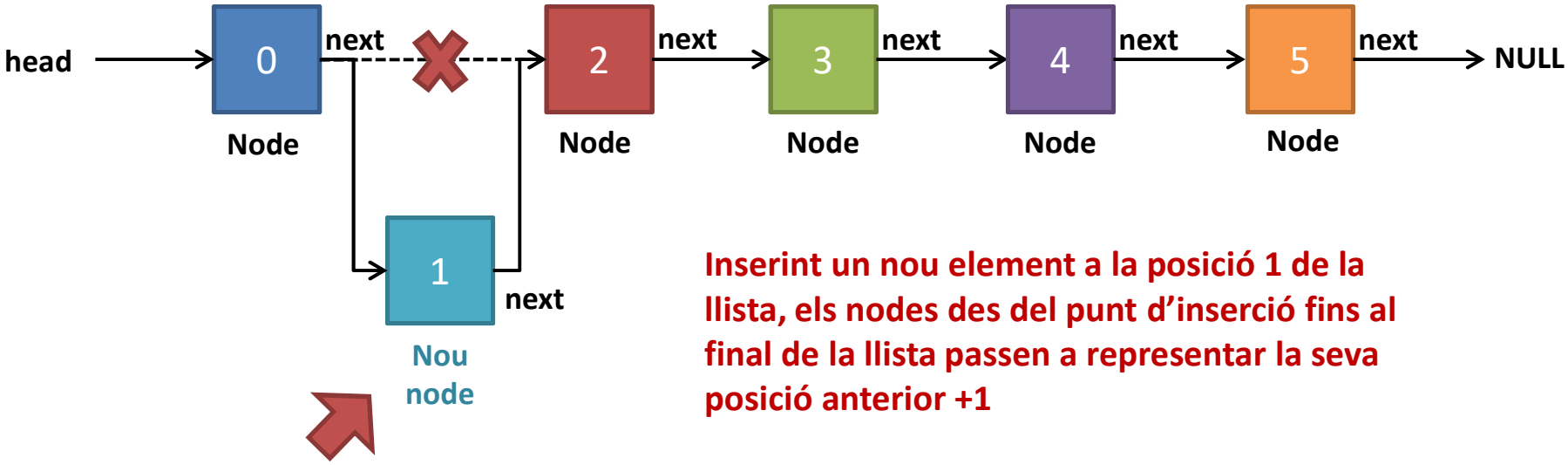
Llista enllaçada (2/4)

- Aquesta forma d'implementar llistes **simplifica molt les operacions d'afegir/inserir/eliminar elements**
- Els seus nodes no estan però emmagatzemats en posicions contigües de memòria. **Haurem de començar des de head i recórrer, node a node, fins a assolir el que representi la posició desitjada**
  - **L'obtenció d'un element depèn de la seva mida** ☹
- Per tal d'afegir un nou element, es crea un nou node que el guardi. Aquest s'afegeix al final de la llista: next del que era el darrer node fins ara l'haurà d'apuntar



Llista enllaçada (3/4)

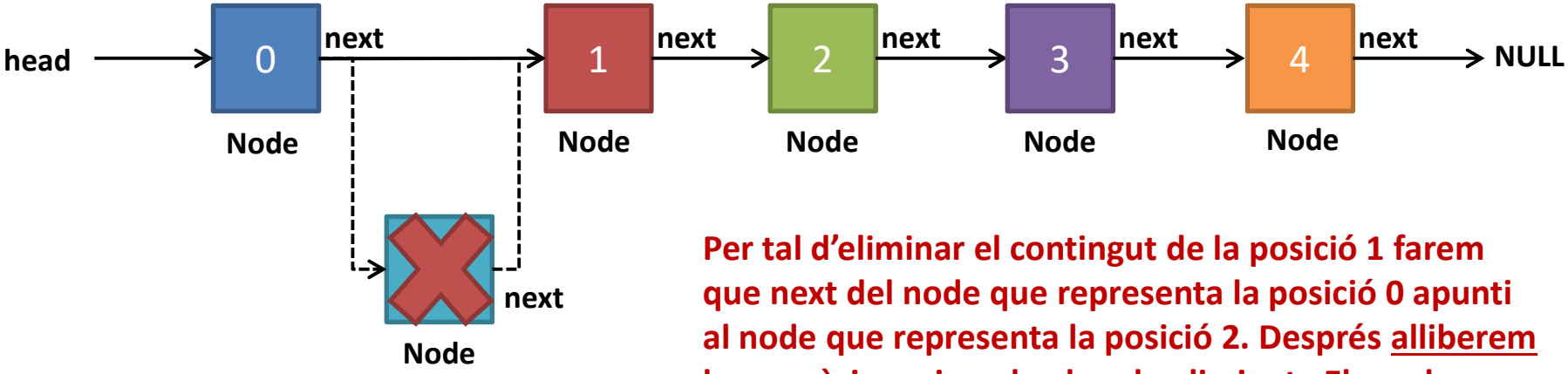
- La **operació d'inserció** d'un element nou a la llista és semblant:
  - Es crea un nou node que el guardi
  - S'aconsegueix que el punter next del nou node apunti al que serà el seu node posterior a la llista (el que representava la posició d'inserció)
  - S'aconsegueix que el punter next del que serà el seu node anterior (el que representava la posició anterior a la d'inserció) apunti al nou node



Inserint un nou element a la posició 1 de la llista, els nodes des del punt d'inserció fins al final de la llista passen a representar la seva posició anterior +1

Llista enllaçada (4/4)

- En el cas de la **operació d'eliminació**:
  - Aconseguim que next del node anterior a la posició d'eliminació apunti al node que representa la posició posterior a la d'eliminació
  - **<u>Alliberem la memòria assignada al node eliminat</u>**



Per tal d'eliminar el contingut de la posició 1 farem que next del node que representa la posició 0 apunti al node que representa la posició 2. Després <u>alliberem la memòria assignada el node eliminat</u> . Els nodes posteriors al node eliminat passen a representar la posició anterior -1

Començant a implementar una llista enllaçada en llenguatge C (1/2)

- Per tal d'implementar una llista enllaçada (d'enters) necessitem, en primer lloc, definir les estructures necessàries:

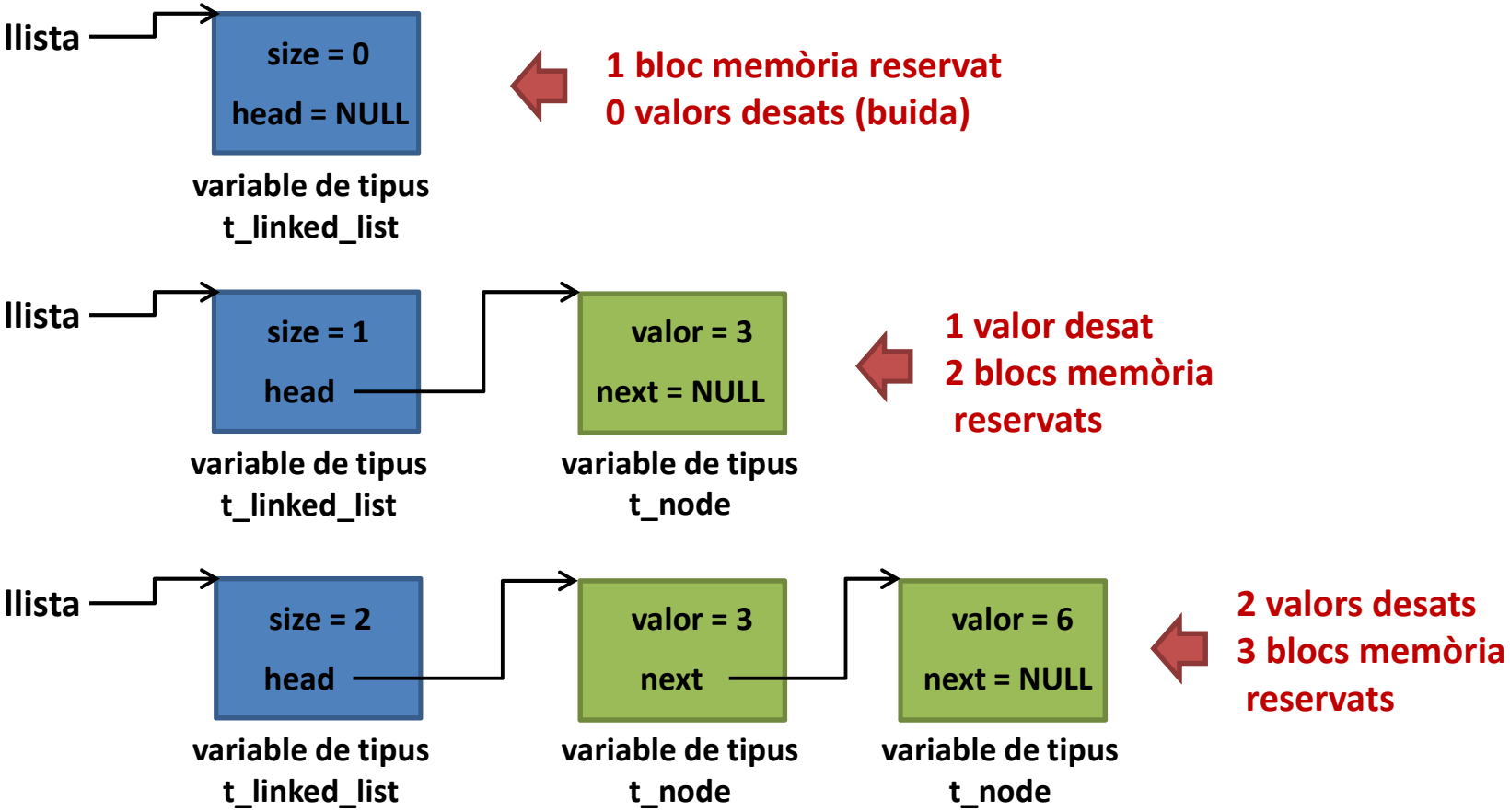
```
typedef struct node{
    int valor;  /*Element guardat*/
    struct node *next;
}t_node;
```

```
typedef struct{
    int size ;
    t_node *head;
}t_linked_list;
```

- En segon lloc, procedirem a implementar operacions semblants a les que vam implementar per a les *array list*:
  - `t_linked_list * crear_llista();`
  - `void alliberar_llista(t_linked_list *llista);`
  - `int add(t_linked_list *llista, int valor);`
  - `void mostrar_llista(t_linked_list *llista);`
  - `etc...`

Començant a implementar una llista enllaçada en llenguatge C (2/2)

- A diferència de les *array list*, la informació continguda en **una llista enllaçada la guardarem en diversos blocs de memòria dinàmica:**



Crear i alliberar llista, afegir element

- Crear una nova llista enllaçada:
  1. Reservar (*malloc*) bloc de memòria per a variable `t_linked_list`
  2. Inicialitzar camps `size = 0` i `head = NULL`
  3. Retornar punter a bloc de memòria reservat
- Alliberar llista enllaçada prèviament creada:
  1. Alliberar (*free*) blocs de memòria que guarden variables `t_node`
  2. Alliberar (*free*) bloc de memòria que guarda variable `t_linked_list`
- Afegir nou valor a llista enllaçada:
  1. Reservar bloc de memòria per a variable `t_node` (nou node)
  2. Inicialitzar camps `valor` i `next = NULL`
  3. Si encara llista buida: aconseguir que `head` apunti a nou node
  4. Sinó: recórrer llista fins a darrer node; el seu `next` apuntarà nou node
  5. Incrementar (+1) camp `size` de variable `t_linked_list`

Exercicis I

- **<u>EXERCICI 4.1</u>**: Comença incloent la declaració dels structs `t_node` i `t_linked_list` al fitxer de capçalera `linked_list.h`. Inclou també la declaració de les capçaleres de funció següents, que després codificarem al fitxer `linked_list.c`:

```
t_linked_list * crear_llista();
void alliberar_llista(t_linked_list *llista);
int add(t_linked_list *llista, int valor);
void mostrar_llista(t_linked_list *llista);
```

- **<u>EXERCICI 4.2:</u>** Implementa la funció `t_linked_list * crear_llista();` aquesta funció ha de reservar memòria dinàmica per a una nova llista enllaçada i inicialitzar-la (buida), retornant un punter a aquesta. En cas de succeir qualsevol problema durant la reserva de memòria dinàmica, la funció retorna NULL.

Exercicis I

- **<u>EXERCICI 4.3</u>**: Implementa al mateix fitxer la funció `void alliberar_llista(t_linked_list *llista);` aquesta funció ha d'alliberar qualsevol bloc de memòria dinàmica prèviament reservat per a la llista apuntada per `llista`.
- **<u>EXERCICI 4.4</u>**: Implementa al mateix fitxer la funció `int add(t_linked_list *llista, int valor);` aquesta funció ha d'afegir el valor enter `valor` al final de la llista apuntada per `llista`. La funció retorna 0 si l'enter s'ha pogut afegir o -1 en cas d'ocórrer qualsevol problema durant la reserva de memòria dinàmica.
- **<u>EXERCICI 4 .5</u>**: Implementa al mateix fitxer la funció `void mostrar_llista(t_linked_list *llista)`; aquesta funció ha de mostrar per pantalla el contingut de la llista apuntada per `llista`, seguint el format `[v_0, v_1, ..., V_N-1]`.

Inserció i eliminació d'elements (1/2)

- Inserir nou valor a llista enllaçada:
  1. Reservar bloc de memòria per a variable `t_node` (nou node)
  2. Inicialitzar el seu camp `valor`
  3. Si posició d'inserció és 0, `next` del nou node serà `head` de la llista fins ara, passant a ser el nou `head` de la llista
  4. Sinó, cercar node que representi la posició anterior a la d'inserció a la llista; inicialitzar `next` del nou node a `next` del node trobat; `next` del node trobat passarà després a ser el nou node
  5. Incrementar (+1) camp `size` de variable `t_linked_list`

Inserció i eliminació d'elements (2/2)

- Eliminar valor de certa posició de llista enllaçada:
  1. Si posició d'eliminació és 0, apuntar a node a eliminar (el primer de la llista); `head` de la llista serà `next` del node a eliminar
  2. Sinó, buscar node anterior a la posició d'eliminació; apuntar node a eliminar; `next` del node trobat passa a ser igual a `next` del node a eliminar
  3. Alliberar bloc de memòria dinàmica prèviament reservat per a node a eliminar
  4. Reduir en una unitat (-1) camp `size` de variable `t_linked_list`

Exercicis I - Continuació

- **<u>EXERCICI 4.6:</u>** Afegeix al fitxer `linked_list.h` les capçaleres de les funcions següents:

```
t_node * obtenir_node(t_linked_list *llista, int index);
int set(t_linked_list *llista, int index, int valor);
int inserir(t_linked_list *llista, int index, int valor);
int eliminar(t_linked_list *llista, int index);
void invertir(t_linked_list *llista);
void ordenar(t_linked_list *llista);
```

Implementa al fitxer `linked_list.c` la funció `t_node * obtenir_node(t_linked_list *llista, int index);` aquesta funció ha de retornar un punter al node que representa la posició `index` de la llista enllaçada apuntada per `llista`. Si la posició `index` és invàlida (és a dir, `<0` o `>=llista->size`), la funció retorna NULL .

Exercicis I - Continuació

- **<u>EXERCICI 4.7:</u>** Implementa al mateix fitxer la funció `int set(t_linked_list *llista, int index, int valor);` aquesta funció ha de substituir el valor ubicat a la posició `index` de la llista apuntada per `llista` pel nou valor passat com a paràmetre. Si el valor ha estat substituït, la funció retorna 0; si la posició `index` és invàlida, la funció retorna -2.

- **<u>EXERCICI 4.8:</u>** Implementa al mateix fitxer la funció `int inserir(t_linked_list *llista, int index, int valor);` aquesta funció ha d'inserir l'enter `valor` a la posició `index` de la llista passada com a paràmetre. Si la posició `index` és invàlida (`<0` o `>llista->size`), la funció retorna -2; en cas de succeïr qualsevol problema durant la reserva de memòria, la funció retorna -1; si la inserció és correcta, la funció acaba retornant 0.

Exercicis I - Continuació

- **<u>EXERCICI 4.9:</u>** Implementa al mateix fitxer la funció `int eliminar(t_linked_list *llista, int index);` aquesta funció ha d'eliminar l'enter ubicat a la posició `index` de la llista passada com a paràmetre. Si la posició `index` és invàlida (és a dir, `<0` o `>=llista->size`), la funció retorna -2; si l'eliminació és correcta, la funció acaba retornant 0.

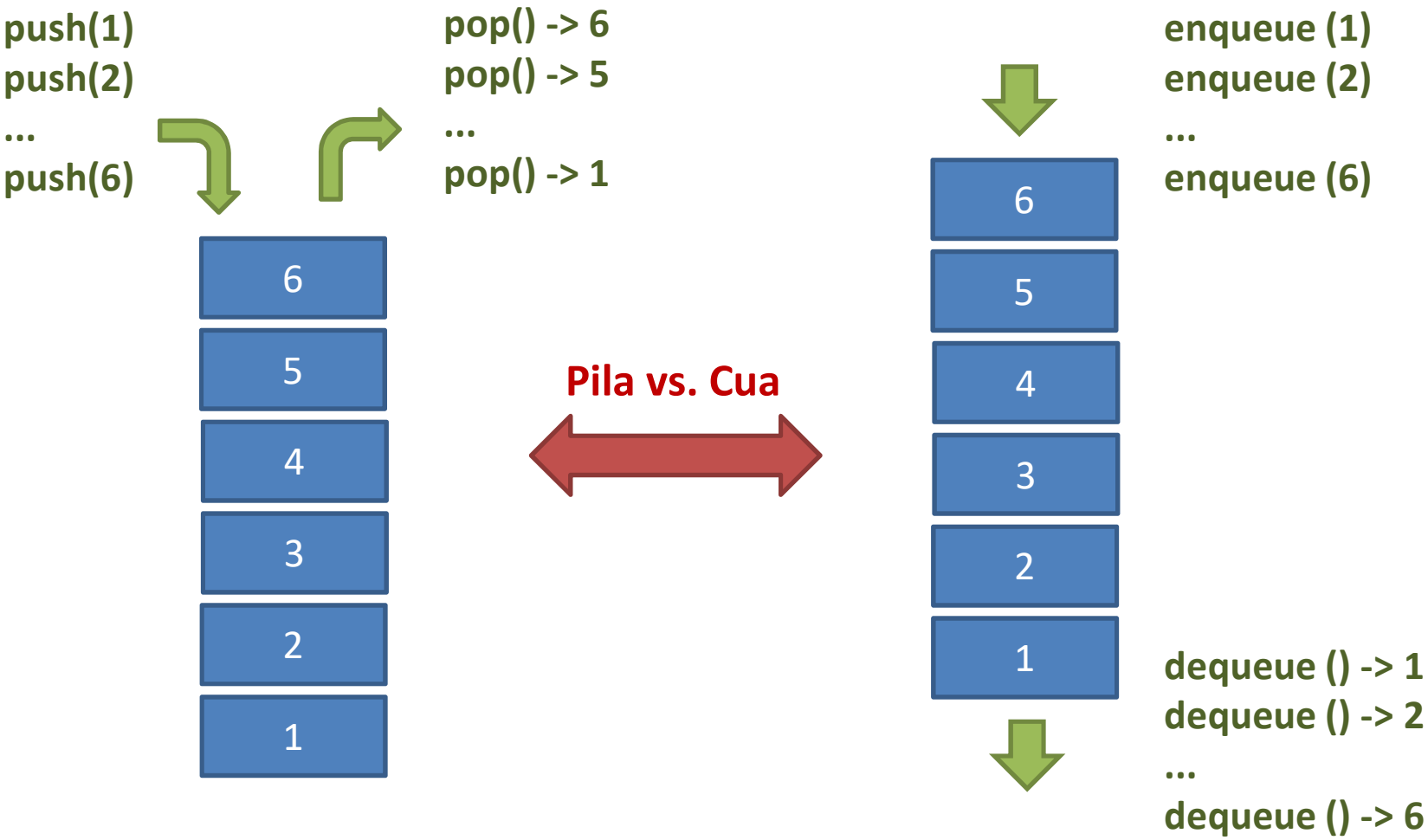
- **<u>EXERCICI 4.10:</u>** Implementa al mateix fitxer la funció `void invertir(t_linked_list *llista);` aquesta funció ha d'invertir el contingut de la llista apuntada per `llista` . Pots usar un vector dinàmic auxiliar per implementar la funcionalitat desitjada.

- **<u>EXERCICI 4.11:</u>** Implementa al mateix fitxer la funció `void ordenar(t_linked_list *llista);` aquesta funció ha d'ordenar la llista apuntada per `llista` mitjançant l'algorisme d'ordenació per selecció directa. Implementa'l sense usar cap vector dinàmic auxiliar.

Piles i cues (1/2)

- Les **piles** i les **cues** són **estructures de dades dinàmiques capaces d'emmagatzemar una seqüència ordenada d'elements**, com les llistes
- Es diferencien d'aquestes pel fet d'oferir una operativa més específica, de gran interès ateses les seves nombroses aplicacions:
  - Les **piles** permeten **emmagatzemar-hi elements**, **extraient-los en mode LIFO** (*Last In, First Out*): el darrer element afegit és el primer que sortirà de la pila
  - Les **cues** permeten **emmagatzemar-hi elements, extraient-los en mode FIFO** (*First In, First Out*): el primer element afegit és el primer que sortirà de la cua

Piles i cues (2/2)



Començant a implementar una pila

- Per implementar una pila podem fer servir una llista enllaçada:

```
typedef struct node{
    int valor; /*Element guardat*/
    struct node *next;
}t_node;
```

```
typedef struct{
    int size;
    t_node *head;
}t_stack;
```

- Haurem d'implementar les operacions bàsiques presentades:
  - `t_stack * crear_pila();`
  - `void alliberar_pila(t_stack *pila);`
  - `int push(t_stack pila, int valor);`
  - `int pop(t_stack *pila, int *res);`

Exercicis II

- **<u>EXERCICI 4.12</u>** : Comença incloent la declaració dels structs `t_node` i `t_stack` al fitxer de capçalera `stack.h`. Inclou també la declaració de les capçaleres de funció següents, que després codificarem al fitxer `stack.c`:

```
t_stack * crear_pila();
void alliberar_pila(t_stack *pila);
int push(t_stack *pila, int valor);
int pop(t_stack pila, int *res);
```

- **<u>EXERCICI 4.13:</u>** Implementa les funcions `t_stack * crear_pila();` i `void alliberar_pila(t_stack *pila);` al fitxer de codi font `stack.c`, basant-te en la implementació de les funcions similars per a llistes enllaçades realitzades als exercicis 4.2 i 4.3.

Exercicis II

- **<u>EXERCICI 4.14:</u>** Implementa també la funció `int push(t_stack *pila, int valor);` aquesta funció ha d'apilar l'enter proporcionat com a paràmetre a la pila apuntada per `pila`. Si es produeix qualsevol problema durant la reserva de memòria dinàmica, la funció retorna -1; sinó la funció acaba retornant 0.
- **<u>EXERCICI 4.15:</u>** Implementa també la funció `int pop(t_stack *pila, int *res);` aquesta funció ha d'extreure i retornar el següent element de la pila apuntada per `pila`. La funció informarà a través del paràmetre `res` de si loperació ha estat satisfactòria o no (pila buida), assignant el valor 0 o -1 a l'enter que apunta.

Exercicis II

- **<u>EXERCICI 4.16</u>** : Comença a implementar el fitxer de capçalera `queue.h` que inclogui la definició dels structs `t_node` i `t_queue` i les següents capçaleres de funcions, que ens permetran implementar una estructura de dades dinàmica de tipus cua:

```
typedef struct node{
   int valor; /*Element guardat*/
   struct node * next;
}t_node;
```

```
typedef struct{
    int size;
    t_node *head;
    t_node *tail;
}t_queue;
```

```
t_queue * crear_cua();
void alliberar_cua(t_queue *cua);
int enqueue(t_queue *cua, int valor);
int dequeue(t_queue *cua, int *res);
```

Exercicis II

- **<u>EXERCICI 4.17:</u>** Implementa les funcions `t_queue * crear_cua();` i `void alliberar_cua(t_queue *cua);` al fitxer de codi font `queue.c`, basant-te en la implementació de les funcions similars per a piles de l'exercici 4.13.

- **<u>EXERCICI 4.18:</u>** Implementa també la funció `int enqueue(t_queue *cua, int valor);` aquesta funció ha d'afegir el valor enter `valor` al final de la cua apuntada per `cua`. Tingues en compte el camp `tail` de l'struct `t_queue` per aquest propòsit. Si es produeix qualsevol problema durant la reserva de memòria dinàmica, la funció retorna -1; sinó acaba retornant 0 .

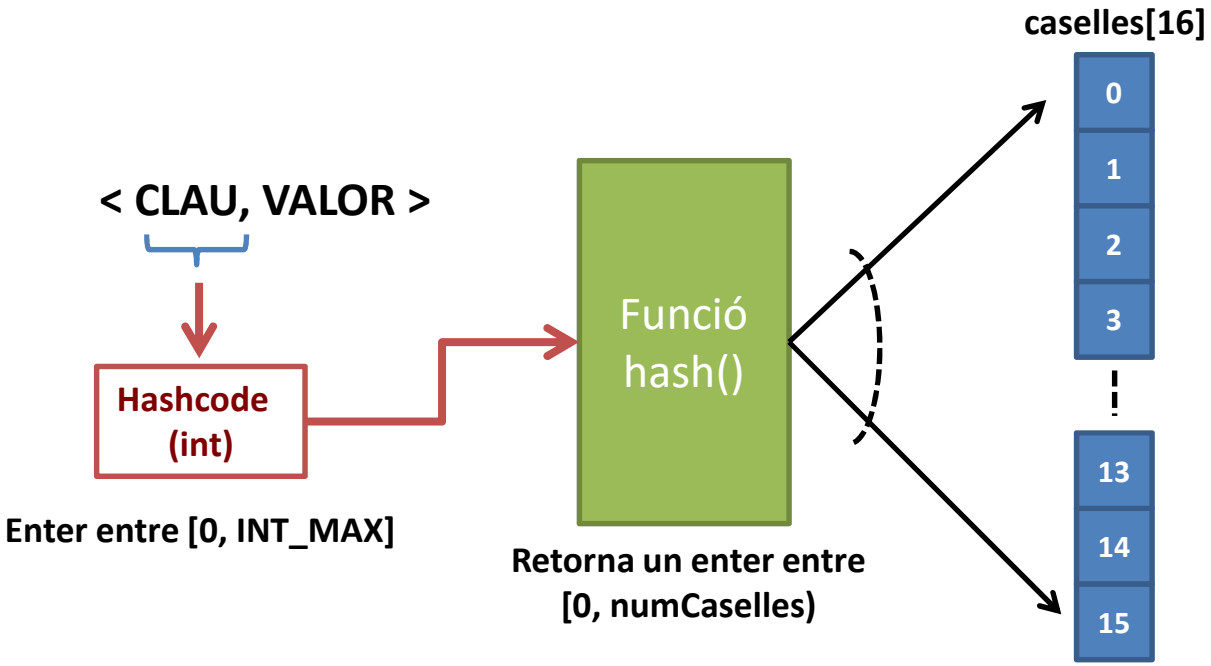
- **<u>EXERCICI 4.19:</u>** Implementa també la funció `int dequeue(t_queue *cua, int *res);` aquesta funció ha d'extreure i retornar el primer element de la cua apuntada per `cua`. La funció informarà a través del paràmetre `res` de si l'operació ha estat satisfactòria o no (cua buida), assignant el valor 0 o -1 a l'enter que apunta.

Taules hash

- Una **taula hash** és una **estructura de dades dinàmica no lineal. Guarda parelles** d'elements **<CLAU, VALOR>**, **on CLAU** actua com a **identificador de VALOR** dins d'ella
- Permet una **cerca eficient d'un VALOR a partir de la seva CLAU**, molt més eficient que una cerca seqüencial en una llista
  - Similar a un diccionari tradicional: CLAU = terme i VALOR = definició
- En una taula hash **les claus han de ser úniques**
- **Molt útils** quan vulguem **desar-hi elements**, als quals els puguem associar una **clau única** que sigui un **criteri desitjat de cerca**
  - Taula hash de parelles <NIF, Persona>, o <ISBN, Llibre>, o <Matrícula, Cotxe>, o <e-mail, Usuari>, etc...

Funcionament d'una taula hash (1/4)

- Una possible **implementació de taula hash** és la **d'adreçament tancat (hashing obert):**
  - La taula hash desa un **array d'un cert nombre de posicions, anomenades caselles**, a determinar quan es crea
  - Donada una **nova parella <CLAU, VALOR>**, la funció hash indica **a quina casella de l'array s'ha de guardar**



Funcionament d'una taula hash (2/4)

- L'etapa d'**obtenció del hashcode a partir de la CLAU** és important **quan la CLAU no és un valor enter comprès entre [0, INT_MAX]**
- **L'elecció de l'estratègia que cal seguir és lliure:** si CLAU és una cadena de caràcters, podriem sumar els codis ASCII de cada caràcter

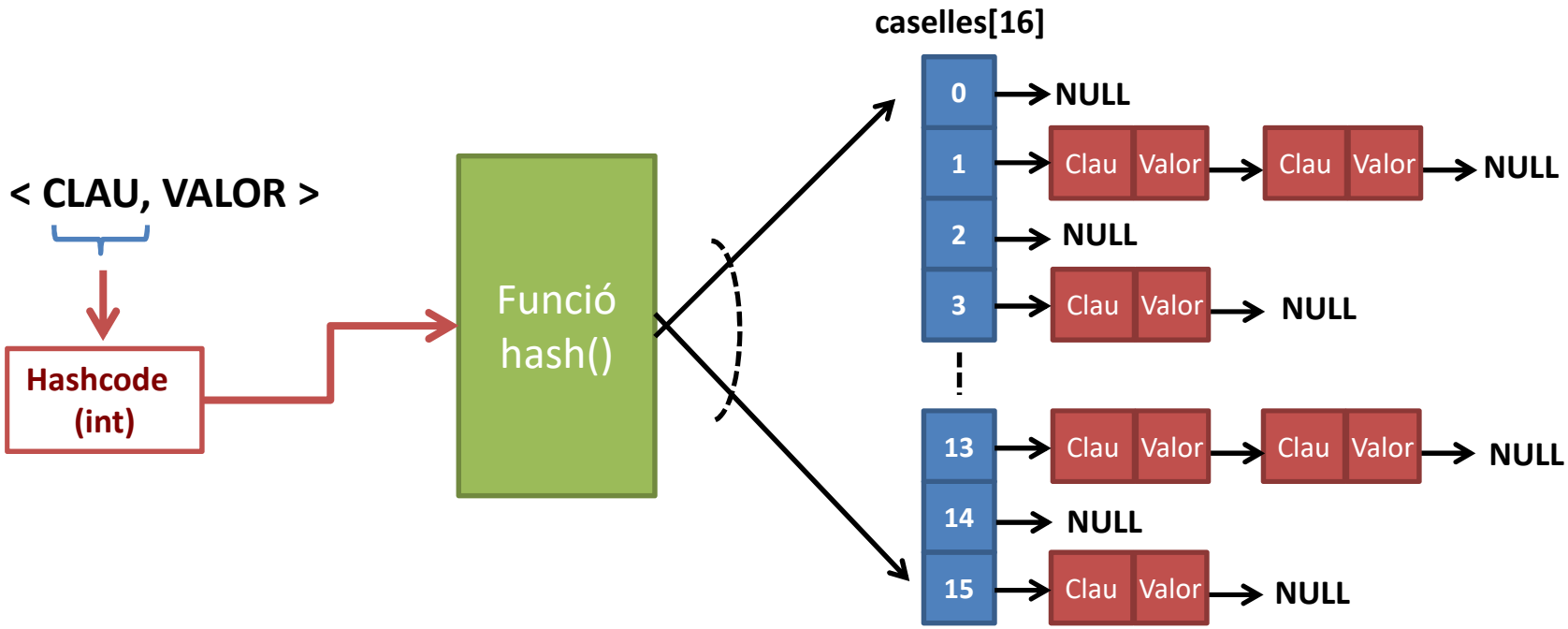
```
int hash_code(char * key){
   int i = 0, hc = 0;
   while(i < strlen(key)){
       hc += key[i]*(i+1);
       i++;
   }
   return(hc);
}
```

- **El mateix succeeix amb la funció hash**: una estratègia bàsica és utilitzar l'operador mòdul (%)

```
int casella = hc % num Caselles;
```

Funcionament d'una taula hash (3/4)

- Les **taules hash resolen col·lisions mitjançant una llista enllaçada per cada casella**: desa totes aquelles parelles <CLAU, VALOR> per a les quals la funció hash ha retornat el mateix resultat



Funcionament d'una taula hash (4/4)

- Les **cerques** sempre seran de **VALOR a partir de CLAU:**
  1. A partir de CLAU generem hashcode i cridem a la funció hash, que retorna la casella on en teoria hi ha VALOR
  2. Recorrem la llista enllaçada buscant la parella <CLAU, VALOR>; retornem VALOR si trobada

- La mida de l'**array intern** és important:
  - **Molt gran**: poc probable que hi hagi col·lisions a la taula hash, però ocupa molt espai en memòria
  - **Massa petit**: ocupa poc espai en memòria però provoca moltes col·lisions que exigeixen cerques seqüencials, disminuint l'eficiència de la taula hash

Exercicis III

- **<u>EXERCICI 4.20</u>** : Comença a implementar el fitxer de capçalera `hashtable.h` de forma que inclogui la definició dels següents structs i capçaleres de funcions, que ens permetran implementar una taula hash de persones identificades pel seu NIF com a clau:

```
typedef struct{
   int num;
   char lletra;
}t_nif;

typedef struct node{
   t_persona p;
   struct node *next;
}t_node;
```

```
typedef struct{
   t_nif nif;
   char nom[20];
}t_persona;

typedef struct{
   int ncas;
   t_node **array_caselles;
}t_hasht;
```

```
t_hasht * crear_taula (int ncas);
void alliberar_taula (t_hasht *taula);
```

Exercicis III

```
int conte_persona(t_hasht *taula, t_nif nif);
int put(t_hasht *taula, t_nif nif, t_persona p);
t_persona * get(t_hasht *taula, t_nif nif);
int eliminar(t_hasht *taula, t_nif nif);
```

- **EXERCICI 4.21:** Implementa les funcions `t_hasht * crear_taula(int ncas);` i `void alliberar_taula (t_hasht *taula);` al fitxer de codi font `hashtable.c`. La primera funció reserva memòria dinàmica per a una nova taula hash de persones amb un array intern de `ncas` caselles, retornant un punter a aquesta, o NULL si succeeix qualsevol problema durant la reserva de memòria dinàmica. La segona funció allibera qualsevol bloc de memòria dinàmica reservat per a la taula hash apuntada per `taula`.

- **EXERCICI 4.22:** Implementa la funció `int conte_persona (t_hasht *taula, t_nif nif);` aquesta funció ha de cercar si existeix la persona amb NIF igual a `nif` a la taula hash apuntada per `taula`. La funció retorna 1 si existeix aquesta o 0 en cas contrari.

Exercicis III

- **<u>EXERCICI 4.23:</u>** Implementa també la funció `int put(t_hasht *taula, t_nif nif, t_persona p);` aquesta funció ha d'emmagatzemar a la taula hash apuntada per `taula` la persona `p` identificada pel NIF `nif` com a clau. Si succeix qualsevol problema de reserva de memòria, la funció retorna -1. Si ja hi havia una altra persona a la taula amb NIF idèntic, la funció acaba retornant -2. Si l'operació conclou satisfactòriament, la funció retorna 0.

- **<u>EXERCICI 4.24:</u>** Implementa també la funció `t_persona * get (t_hasht *taula, t_nif nif);` aquesta funció ha de retornar un punter a la persona amb NIF `nif` guardada a la taula hash apuntada per `taula`. Si aquesta persona no es troba a la taula, la funció retorna NULL .

- **<u>EJERCICI 4.25:</u>** Implementa també la funció `int eliminar (t_hasht *taula, t_nif nif);` aquesta funció ha d'eliminar de la taula hash apuntada per `taula` la persona amb NIF igual a `nif`. Si no és troba, la funció retorna -2. Si la operació és satisfactòria, la funció retorna 0.